n8n orchestration overview

An important part of the project involves agentic AI orchestration using n8n. Explain how n8n is configured in the local network. On which computer will it run in a self-hosted environment?
Given what you understand about project Reachy_Local_08.4.2, explain at a high level how n8n will orchestrate the project.

Thought for 35s >

Short version: n8n lives on **Ubuntu-1** and acts as the traffic controller for the whole "video → label → train → evaluate → deploy" loop. It never runs on Jetson; Jetson just listens for new engines and does real-time inference.

I'll break it into two pieces:

---

1. Where n8n runs and how it sits on the LAN

From your own requirements:

- n8n is explicitly "**agent orchestration on Ubuntu 1**." requirements_08.4.2
- The AGENTS spec says: "All orchestration occurs in **n8n running on Ubuntu 1**." AGENTS_08.4.2
- LAN IPs are fixed:
  - Ubuntu 1: **10.0.4.130** (training + storage + Media Mover + Postgres)
  - Ubuntu 2: **10.0.4.140** (web UI, Nginx, API gateway)
  - Jetson NX: **10.0.4.150** (DeepStream runtime) requirements_08.4.2

So in the self-hosted setup:

- **n8n server process**
  - Runs on **Ubuntu-1** (10.0.4.130), probably as a Docker container or systemd-managed service.
  - Binds to a local port (e.g., 5678) on that host.
  - Is only reachable from inside the LAN; you don't expose it to the wider internet.

- **Local connectivity from n8n** (all within 10.0.4.x):
  - Talks to **Postgres 16** locally on Ubuntu-1 for all metadata, queue-ish behavior, and audit logs. AGENTS_08.4.2
  - Calls the **Media Mover FastAPI** on Ubuntu-1 for ingest/promote/manifest rebuild.
  - Calls **MLflow** and shell/command nodes to launch TAO containers (training + export) on Ubuntu-1. requirements_08.4.2
  - Reaches **Ubuntu-2** (10.0.4.140) via HTTP/Webhook nodes when it needs to notify the UI or kick off something from the UI side.
  - Reaches **Jetson NX** (10.0.4.150) via SSH/HTTP/SCP style actions when the Deployment Agent pushes a new `.engine` and restarts DeepStream. AGENTS_08.4.2

Mentally: Ubuntu-1 is the **"backend brain + heavy compute"** box, and n8n is the **conductor** sitting there with direct wires to DB, TAO, Media Mover, and the other two nodes.

---

2. How n8n orchestrates Reachy_Local_08.4.2 (high-level loop)

The requirements define **nine agents** in an "agentic AI system," and explicitly say they're orchestrated by n8n on Ubuntu-1. requirements_08.4.2
The AGENTS file then fleshes out each agent's responsibilities. AGENTS_08.4.2

Think of each "Agent" as a **named n8n workflow (or set of workflows)** with a very narrow job. Together, they run the end-to-end lifecycle:

2.1 Big picture: the loop

Goal loop:
`video created → ingested → labeled/promo'd → manifests rebuilt → training → evaluation → deployment → monitoring/privacy`

n8n is what actually glues those phases together:

1. **Agent 1 – Ingest (n8n workflow)** AGENTS_08.4.2
   - Trigger: Webhook from Ubuntu-2 UI or a generation job finishing.
   - n8n steps:
     - HTTP → call Media Mover to move/upload file into `/videos/temp/`.

    - Function node → compute/verify `sha256`, assemble metadata.
    - Postgres node → insert a new row in `media.video` with paths, fps, resolution, hash.
    - Command/HTTP node → tell thumbnail service to create a JPG in `/thumbs/`.
    - Finally emit an `ingest.completed` event (could be a row in DB + notify UI).
2. **Agent 2 – Labeling (n8n workflow)** AGENTS_08.4.2
    - Trigger: User labels a clip as `happy` / `sad` from the UI (webhook back to n8n or direct to Media Mover + n8n listens).
    - n8n:
        - Writes/updates label info in Postgres (for train split only).
        - Checks constraints like `chk_split_label` and keeps per-class counts.
        - Computes "are we close to a 50/50 balance?" and updates counters that UI reads.
3. **Agent 3 – Promotion / Curation** AGENTS_08.4.2
    - Trigger: Labeling Agent or UI asks to "promote" a clip.
    - n8n:
        - Calls Media Mover `/api/promote` to atomically move files `temp → train` or `temp → test`.
            - For `train`: label *must* be set.
            - For `test`: label *must* be NULL (you enforce that all the way down the stack). requirements_08.4.2
        - Inside the same workflow, uses Postgres node to update the `split` field transactionally.
        - Updates cached class ratios and emits "ready/not ready" status back to the UI.
4. **Agent 4 – Reconciler / Audit** AGENTS_08.4.2
    - Trigger: Cron (periodic) or manual button from UI.
    - n8n:
        - Walks the filesystem (via Command nodes) and compares hashes/paths with Postgres.
        - Detects missing/orphaned files, duplicates, or drift.

    - Rebuilds `train_manifest.json` and `test_manifest.json` if anything changed.
    - Pushes a summary report into metrics / logs (and exposes it to Grafana via Prometheus).
5. **Agent 5 – Training Orchestrator** AGENTS_08.4.2
    - Trigger: Dataset reaches thresholds – e.g., 50/50 balance, minimum per-class count. Requirements reference that as part of Gate A. requirements_08.4.2
    - n8n:
        - Checks DB counts and manifest integrity first (Postgres node).
        - When conditions are met, runs a Command node that starts the **TAO 4.0 EmotionNet training container** with the pinned `emotion_train_2cls.yaml`, mounting `/videos/train/` as the dataset. requirements_08.4.2
        - Scrapes logs/metrics and posts them to MLflow (HTTP or Python node).
        - Writes a `training.completed` record into Postgres with dataset hash, TAO version, F1, etc.
6. **Agent 6 – Evaluation** AGENTS_08.4.2
    - Trigger: Training Orchestrator emits `training.completed` with good enough metrics and test set is balanced.
    - n8n:
        - Runs a separate TAO/DeepStream validation job against the **validation split**, never touching `videos/test/` labels directly.
        - Computes confusion matrix, accuracy, and Gate A/B metrics.
        - Logs results to Postgres + MLflow, and marks the engine as “candidate for deployment.”
7. **Agent 7 – Deployment** AGENTS_08.4.2
    - Trigger: Human click “approve deploy” in UI → webhook → n8n.
    - n8n:
        - Copies the exported `.engine` from Ubuntu-1 to Jetson (`/opt/reachy/models/emotion.engine`) using SSH/SCP nodes.
        - Updates the DeepStream config and restarts the DeepStream service (Command node on Jetson).

    - Probes metrics on Jetson (HTTP/metrics endpoint) to ensure FPS ≥ target and latency within bounds.
    - If metrics regress, it rolls back to the previous engine.
    - Logs deployment metadata (engine version, FPS, latency, timestamp) to Postgres.
8. **Agent 8 – Privacy / Retention** AGENTS_08.4.2
    - Trigger: Cron (e.g., nightly) + manual “purge” from UI.
    - n8n:
        - Finds `temp/` videos older than TTL (say 7 days) and deletes them from disk.
        - Updates Postgres to mark them purged.
        - Emits `privacy.purged` events; if something goes wrong, it emits `privacy.violation`.
9. **Agent 9 – Observability / Telemetry** AGENTS_08.4.2
    - Trigger: Cron (e.g., every minute/5 minutes).
    - n8n:
        - Pulls metrics from other agents (queue depth, failure rates, training duration, dataset counts).
        - Pushes them into Prometheus and surfaces dashboards in Grafana.
        - If error budgets or latencies blow past your SLOs, it sends alerts and logs an `obs.snapshot`.

There’s also an **optional Generation Balancer sub-agent** that can bias future synthetic video prompts if one class gets under-represented. AGENTS_08.4.2

---

3. How to think about n8n’s role overall

If I boil it down:

- **Jetson** = real-time emotion classification (DeepStream + EmotionNet engine).
- **Ubuntu-2** = human-facing UI for generating, labeling, and monitoring.
- **Ubuntu-1 + n8n** = **all the slow, careful thinking**:
    - Decides when a clip is “in” or “out” of the dataset.
    - Decides when you’ve got enough balanced data to train.

- Decides when a model is good enough to test on Jetson.
- Decides when a Jetson engine is safe to roll out or roll back.
- Makes sure privacy, balance, and auditability rules are actually enforced.

So when you talk about “agentic AI orchestration using n8n” for Reachy_Local_08.4.2, what you’re really saying is:

> *“The robots do the live interaction, but n8n is the **governor** that manages data, training, and deployments under strict rules, end-to-end, on my own hardware.”*

Which is exactly the kind of story a reviewer or hiring manager wants to hear.

If you want, next step we can sketch one concrete n8n workflow (say, Training Orchestrator) node-by-node so it feels less abstract and more “I can build this in an afternoon.”

Note: I have a separate workstation I planned to use for n8n orchestration. You mentioned that the project requirements specify n8n will be installed on Ubuntu 1. My understanding is that it would be better to offload the n8n orchestration on a separate machine since Ubuntu 1 has so much responsibility and heavy workload. Do you agree with this assessment? Should I use n8n on a separate machine dedicated for that sole purpose?
Or, are there benefits to having n8n installed on Ubuntu 1 because it's so central to the overall project? It could be simpler to monitor certain activities directly on Ubuntu 1. What is your opinion?

Thought for 17s >

Yeah, your instinct here is solid: **Ubuntu-1 already has a lot on its plate.** You're not wrong to question stacking n8n on top of that.

Let's walk through it without overcomplicating things.

---

1. The "spec says Ubuntu-1" thing

The docs saying "n8n runs on Ubuntu-1" is a **design choice**, not a law of nature. That was us optimizing for:

- fewer machines to maintain
- easy local calls (Postgres, Media Mover, TAO)
- simpler first implementation

You're allowed to change your mind and bump the orchestrator to a separate box. If you do, we just update `AGENTS_08.4.2.md` and the diagrams so they say "Orchestration host" instead of "Ubuntu-1."

So you're not "breaking the spec"; you're iterating the architecture.

---

2. n8n on Ubuntu-1: pros and cons

Pros

1. Locality = simplicity

- n8n can just run `docker run ...` or `bash` commands for TAO directly on Ubuntu-1.
- No SSH hops, no extra credentials, no “remote command” indirection.
- File paths are local (`/media/.../videos/train`) so manifest rebuilds and audits are trivial.

2. Fewer moving parts

- One less box to patch, name, back up, and remember.
- For early development, it’s nice to just open the browser to `http://10.0.4.130:5678` and see everything where the data lives.

Cons

1. Blast radius

- When you reboot Ubuntu-1 for a kernel/driver update, **you lose**:
  - Postgres
  - Media Mover
  - TAO training
  - MLflow
  - and **n8n**
- If Ubuntu-1 gets hammered by a TAO job and pegs CPU/IO, the orchestrator UI and scheduled workflows can get sluggish too.

2. “Everything important on one ship”

- From a “production-minded engineer” perspective, it’s a little meh that the **control plane** and the **data/compute plane** are fused.
- It *works*, but it’s less impressive than:

> *“I have a dedicated orchestration node that coordinates multiple services across the LAN.”*

---

3. n8n on a separate orchestration workstation

Let’s call it **Orch-1** for sanity.

Pros

1. Clear control-plane / data-plane split

- Orch-1: n8n, alerts, maybe Grafana/Prometheus frontends.
- Ubuntu-1: Postgres, Media Mover, TAO, MLflow, storage.
- Ubuntu-2: Web UI / API.
- Jetson: inference.

That's how real MLOps stacks often look: **one brain coordinating multiple workers**.

2. Better resilience

- If Ubuntu-1 reboots for CUDA drivers, Orch-1 is still up:
  - It can see "Ubuntu-1 is down, training job lost" and log/alert that.
  - Cron-style jobs (privacy/retention, metrics collection, etc.) at least fail in a controlled way instead of vanishing.

3. Room to grow

- If later you add:
  - another project (speech emotion, linguistics pipelines, whatever),
  - another GPU server,
  - or a second Jetson,

n8n on Orch-1 becomes a **shared control surface**: one orchestrator talking to N machines.

4. Good story for reviewers/hiring

You can say:

> *"I separated orchestration onto its own host so the control plane stays stable even when the GPU box is under heavy training load or being upgraded."*

That's the sort of sentence that makes infra people quietly nod.

Cons

1. More wiring

- n8n will now talk over the LAN for everything:
  - Postgres on Ubuntu-1
  - Media Mover API on Ubuntu-1
  - UI webhooks on Ubuntu-2

    - SCP/SSH to Jetson
- For filesystem-based things (manifest rebuilds, audits), n8n won't run `ls` directly on the data disk; it'll:
    - SSH into Ubuntu-1
    - run scripts **there**
    - parse results back into n8n

Totally doable, but slightly more setup: SSH keys, hostnames, secrets management.

2. Another box to babysit

- OS updates, backups, monitoring for Orch-1 itself.
- Small overhead, but it's more mental surface area.

---

4. So… what should *you* do?

Given **your situation specifically**:

- You **already** have:
    - Ubuntu-1: heavy ML + DB + storage
    - Ubuntu-2: web/app layer
    - A **separate workstation** you intended for n8n
- Your long-term goal is:

> *"Look like someone who designs production-ish, scalable systems, not just a single machine demo."*

I'd lean to:

✅ Recommendation

Yes, use the separate workstation as a dedicated orchestration host for n8n.

Treat it as your **control plane**:

- **Orch-1 (n8n)**:
    - Runs n8n (Docker or bare-metal).
    - Stores n8n's own DB (SQLite or Postgres).
    - Knows how to:
        - Talk to Postgres on Ubuntu-1

    - Call Media Mover on Ubuntu-1
    - Call the UI / API on Ubuntu-2
    - SSH / SCP to Jetson and Ubuntu-1 for scripts and deployments
- **Ubuntu-1** keeps all the heavy lifting but becomes a “worker” from n8n’s perspective.

The performance hit is negligible on a gigabit LAN; n8n is mostly **IO and coordination**, not a CPU hog.

---

5. Practical path forward (so this doesn’t become a yak farm)

To keep friction low:

1. **Design as if Orch-1 exists now**, even if you prototype on Ubuntu-1 for a week.
2. In your config/env:
    - Don’t hard-code “localhost”; always use:
        - `DB_HOST=10.0.4.130` (Ubuntu-1)
        - `MEDIA_MOVER_URL=http://10.0.4.130:8000`
        - `UI_GATEWAY_URL=http://10.0.4.140:8080`
        - `JETSON_HOST=10.0.4.150`
3. That way:
    - If you start with n8n on Ubuntu-1, it works.
    - When you’re ready to move n8n to Orch-1, you just:
        - install n8n there,
        - point it at those same IPs,
        - and you’re done. No logic change; just where it runs.

So: your “this feels like too much on Ubuntu-1” spidey-sense is valid.
If you’re willing to manage one more box, **dedicated n8n on a separate workstation is the cleaner, more “production-minded” architecture.**

If you want next, we can sketch the exact env vars + network diagram for the “Orch-1 control plane” so it’s ready to drop into your docs without rewriting everything.